

Relatório de Integração — MOM

Resumo: Neste projeto adotei um Message-Oriented Middleware (MOM) para introduzir comunicação assíncrona entre a API HTTP e o worker de processamento, mantendo o banco de dados relacional como fonte da verdade. O objetivo foi desacoplar a publicação de eventos do processamento, melhorar a robustez frente a picos de carga e permitir processamento eventual sem transformar a aplicação em microserviços.

Escolha da ferramenta: RabbitMQ.

Motivo: maturidade, suporte a padrões AMQP, facilidade de execução local via Docker Compose e painel de gerenciamento (Management UI) para depuração. Boa combinação entre recursos avançados (exchanges, bindings, filas duráveis) e simplicidade para um projeto acadêmico. Padrão arquitetural adotado: Publicação/Assinatura (pub/sub) orientada a eventos dentro de um modelo EDA (Event-Driven Architecture).

Implementação prática: a API persiste a mudança no banco e, após commit bem-sucedido, publica um evento (ex.: fila.criada, fila.chamada, fila.finalizada) em uma exchange; um worker (consumer) consome as filas correspondentes (fila.criada.queue, fila.chamada.queue, fila.finalizada.queue) e realiza ações secundárias/logs.

Racional: garante que a API não bloqueie aguardando processamento, e que efeitos colaterais sejam tratados de forma resiliente.

Decisões de integridade e confiabilidade:

- Persistência no banco como fonte da verdade: mensagens representam eventos derivados de mudanças já confirmadas no banco, evitando inconsistências.
- Publicação pós-commit: só publicar após sucesso do persist; reduz risco de eventos fantasma.
- Acknowledgements e re-tries: consumer faz ack manual após processamento; em falha a mensagem é reencaminhada (at-least-once). Aplicação projetada para tolerar duplicidade (idempotência simples via checagem de estado no DB).
- Durability: filas e mensagens configuradas como duráveis em ambiente de produção para sobreviver a reinícios do broker.

Desafios encontrados:

- Gestão de duplicidade e idempotência: como o broker garante ao menos uma entrega, foi necessário projetar handlers idempotentes e checar estado atual no banco antes de aplicar efeitos.
- Orquestração local e observabilidade: acostumar-se ao fluxo de mensagens exigiu uso do Management UI e logs claros ([PUBLISHER], [CONSUMER]).
- Falhas do broker: garantir que a API continue operando se o RabbitMQ estiver indisponível (a publicação é tratada como efeito colateral; API retorna sucesso enquanto o publisher faz log de falha).
- Ordem de entrega e consistência temporal: mensagens podem ser reordenadas entre filas; a lógica não depende de ordering estrito entre eventos diferentes.
- Tratamento de mensagens não processáveis: decisão provisória de logar e re-tentar; melhorias futuras incluem Dead Letter Queues (DLQ) para mensagens inválidas.

Trade-offs e limitações:

- Simplicidade vs. robustez: optou-se por uma implementação leve (worker separado) em vez de um sistema distribuído completo para manter o escopo da sprint.
- Garantia de entrega: adotamos at-least-once por simplicidade; exactly-once exigiria infraestrutura adicional.

Possíveis melhorias futuras:

- Implementar DLQs, backoff exponencial e contadores de tentativas; adicionar esquema de versionamento/avro/json-schema para eventos; métricas e tracing distribuído para rastrear fluxos de eventos; testes de falha simulando broker down.